Assignment *Web Scraping and Trie-Based Search Application*

Problem Statement

Objective:
To develop a Spring Boot web application that scrapes real-time data from specified websites and efficiently stores and retrieves keywords using a trie data structure, allowing users to conduct quick and relevant searches.

This project will leverage the capabilities of Spring Boot for building a robust web application that automates data extraction and provides efficient search functionality using a trie data structure. The application will enhance user access to real-time information and support data-driven decision-making processes.

Background:

In the age of information, businesses require timely access to data from various online sources for competitive analysis, market research, and decision-making. Manual data collection methods are not only labor-intensive but also prone to error. Automating the process of web scraping and providing fast search capabilities can significantly enhance productivity and data accessibility.

Requirements:

1. **Web Scraping Module**:
    - Create a service that can scrape content from user-defined URLs based on specified keywords.
    - Support scheduled scraping at regular intervals or on-demand as per user request.
    - Store the scraped data along with metadata (e.g., source URL, timestamp).
2. **Trie Data Structure**:
    - Implement a trie data structure to index keywords for efficient retrieval.
    - Allow prefix-based search and autocompletion features for enhanced user experience.
3. **API Endpoints**:
    - Provide a RESTful API for users to initiate scraping, perform searches, and manage scraped data.

API Design

1. Scraping Endpoint

- **Endpoint**: /api/v1/scrape
- **Method**: POST

- **Description**: Initiates the web scraping process for specified URLs and keywords.

Example Request URL:

bash

```
POST /api/v1/scrape
```

Request Body:

```
{
  "urls": [
    "https://example.com/news",
    "https://example2.com/blog"
  ],
  "keywords": ["technology", "innovation"],
  "schedule": "2024-11-01T10:00:00Z"
}
```

Sample Response:

```
{
  "status": "success",
  "message": "Scraping initiated successfully.",
  "jobId": "12345",
  "scheduledAt": "2024-11-01T10:00:00Z"
}
```

2. Search Endpoint

- **Endpoint**: /api/v1/search
- **Method**: POST
- **Description**: Performs a keyword search through the scraped data using a trie structure.

Example Request URL:

bash

```
POST /api/v1/search
```

Request Body:

```
{
  "prefix": "tech",
  "limit": 5
}
```

Sample Response:

```
{
  "status": "success",
  "results": [
    {
      "url": "https://example.com/news",
      "matchedContent": "Latest technology trends are shaping the future...",
      "timestamp": "2024-10-29T14:30:00Z"
    },
    {
      "url": "https://example2.com/blog",
      "matchedContent": "The innovation in tech is accelerating...",
      "timestamp": "2024-10-29T12:00:00Z"
    }
  ]
}
```

3. Status Check Endpoint

- **Endpoint**: /api/v1/status/{jobId}
- **Method**: GET
- **Description**: Retrieves the status of a specific scraping job.

Example Request URL:

bash

```
GET /api/v1/status/12345
```

Sample Response:

```
{

  "status": "completed",
  "jobId": "12345",
  "urlsScraped": [
    "https://example.com/news",
    "https://example2.com/blog"
  ],
  "keywordsFound": ["technology", "innovation"],
```

```
  "dataSize": "2 MB",
  "finishedAt": "2024-11-01T10:15:00Z"
}
```


Few things to take care of while submission of code:

- The evaluation of the assignment will be based on code quality
- REST Compliant APIs, (No UI)
- Testable by Postman, (No UI)
- Will prefer classes properly refactored and following design patterns,
- Will Prefer TDD, JUnit